Задание 1-3 (Рабочая тетрадь 2)

Задание 1 (Рабочая тетрадь 2)

Задача: Напишите программу, которая переносит функциональность, использующую Windows API, на Linux.
В исходном коде используются функции `CreateFile` и `ReadFile`.
Ваша задача — заменить эти функции на их эквиваленты для Linux, такие как `open` и `read`.

Реализация

Для автоматического портирования кода с Windows на Linux разработан транслятор на Python, использующий **LLM (qwen3-coder:30b)** через локальный сервер **Ollama**.

Программа [API_translator(Win-Lin).py]: - Читает файлы с Windows-специфичным кодом из папки `InputCode`. - Формирует промпт с жёсткими правилами замены Windows API на POSIX-аналоги. - Передаёт код LLM для переписывания. - Извлекает полученный код и (опционально) проверяет его компиляцию с помощью `g++ -fsyntax-only`. - Сохраняет готовый Linux-совместимый код в папку `OutputCode`.

Ключевые возможности: - Замена `Sleep` → `usleep/nanosleep`, `CreateThread` → `pthread_create`, `VirtualAlloc` → `mmap`, `HANDLE` → `pthread_t` / `int` и т.д. - Поддержка до 3 попыток при ошибках компиляции (LLM получает обратную связь). - Использование `new(std::nothrow)` для безопасного выделения памяти (при необходимости). - Работа как на Windows, так и на Linux (программа написана на Python, требует только установленной Ollama и компилятора g++ для проверки).

Принцип работы

1. **Подготовка окружения**
2. Убедитесь, что запущен сервер Ollama и загружена модель `qwen3-coder:30b`.

3. Создайте папку `InputCode` в директории со скриптом и поместите туда `.cpp` или `.c` файлы, содержащие Windows API.

4. **Запуск транслятора**
   `bash python API_translator(Win-Lin).py`

5. **Логика обработки одного файла**

6. Считывается исходный код.
7. Формируется промпт с правилами замены (см. метод `_build_prompt`).
8. LLM возвращает переписанный код.
9. Из ответа извлекается только код (удаляются markdown-блоки).
10. Если включена верификация (`verify_compile=True`), вызывается `g++ -fsyntax-only` для проверки синтаксиса.
11. При ошибке компиляции она передаётся в следующий промпт (feedback).

12. После успеха код сохраняется в `OutputCode` с тем же именем файла.

13. **Требования к модели**

14. Модель должна хорошо понимать C/C++ и различия Windows/Linux API.

15. В тестах использовалась `qwen3-coder:30b` с температурой `0.1` и контекстом `16384`.

16. **Результат**

17. Все вызовы `CreateFile`, `ReadFile`, `WriteFile`, `CloseHandle` и т.п. заменяются на `open`, `read`, `write`, `close`.
18. Добавляются необходимые заголовки: `<unistd.h>`, `<fcntl.h>`, `<pthread.h>`, `<sys/mman.h>`.

Листинг кода

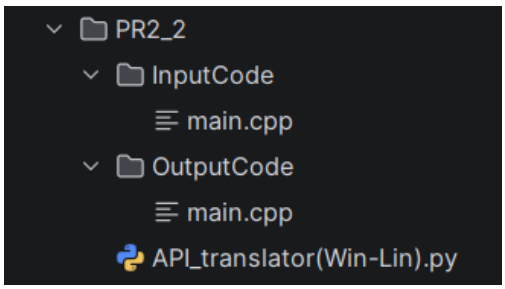
Исходный код транслятора: [API_translator(Win-Lin).py]

```python
import requests
import re
import os
import subprocess
import tempfile
from typing import Optional

class WinToLinuxTranslator:
    def __init__(self, model: str = "qwen3-coder:30b", ...):
        # ... (полный код приведён в файле)

    def _build_prompt(self, source_code: str, error_feedback: Optional[str] = None) -> str:
        # Формирует промпт с правилами замены Windows API
        ...

    def _check_compilation(self, code: str) -> Optional[str]:
        # Проверяет синтаксис через g++ -fsyntax-only
        ...

    def _call_llm(self, prompt: str) -> str:
        # Отправляет запрос к Ollama
        ...

    def translate(self, source_code: str) -> str:
        # Цикл попыток с учётом ошибок компиляции
        ...

def main():
    # Обход всех файлов в InputCode
    ...

if __name__ == "__main__":
    main()
```

Пример использования

Структура папок перед запуском

```
PR2_2/
├── API_translator(Win-Lin).py
├── InputCode/
│   └── windows_example.cpp
└── OutputCode/
```



Входной файл [main.cpp]

```cpp
#include <windows.h>
#include <iostream>
#include <vector>

int main() {
    // Путь к файлу (LPCSTR - Long Pointer to Constant STRing)
    LPCSTR fileName = "data.txt";

    // 1. Создание/Открытие файла
```

```
    // Аналог в Linux: open()
    HANDLE hFile = CreateFileA(
        fileName,               // Имя файла
        GENERIC_READ,           // Режим доступа: только чтение
        FILE_SHARE_READ,        // Совместный доступ
        NULL,                   // Атрибуты безопасности
        OPEN_EXISTING,          // Открыть только если файл существует
        FILE_ATTRIBUTE_NORMAL,  // Стандартные атрибуты
        NULL                    // Шаблонный файл
    );

    if (hFile == INVALID_HANDLE_VALUE) {
        std::cerr << "Ошибка CreateFile. Код: " << GetLastError() << std::endl;
        return 1;
    }

    // Подготовка переменных для чтения
    const DWORD BUFFER_SIZE = 1024;
    char buffer[BUFFER_SIZE];
    DWORD bytesRead = 0;

    // 2. Чтение файла
    // Аналог в Linux: read()
    BOOL result = ReadFile(
        hFile,                  // Хэндл файла
        buffer,                 // Буфер для данных
        BUFFER_SIZE - 1,        // Сколько байт прочитать
        &bytesRead,             // Сколько байт прочитано по факту
        NULL                    // Структура для асинхронного ввода-вывода
    );

    if (result) {
        buffer[bytesRead] = '\0'; // Терминирующий ноль для строки
        std::cout << "Прочитано байт: " << bytesRead << std::endl;
        std::cout << "Данные: " << buffer << std::endl;
    } else {
        std::cerr << "Ошибка ReadFile. Код: " << GetLastError() << std::endl;
    }

    // 3. Закрытие хэндла
    // Аналог в Linux: close()
    CloseHandle(hFile);

    return 0;
}
```

Запуск транслятора

```
cd PR2_2
python API_translator(Win-Lin).py
```

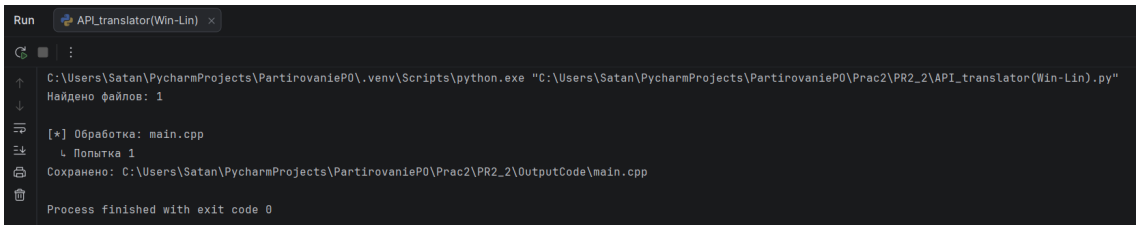
Выходной файл [main.cpp]

```
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <iostream>
#include <vector>

int main() {
    // Путь к файлу
    const char* fileName = "data.txt";

    // 1. Создание/Открытие файла
    int fd = open(fileName, O_RDONLY);
```

```
    if (fd == -1) {
        std::cerr << "Ошибка open. Код: " << errno << std::endl;
        return 1;
    }

    // Подготовка переменных для чтения
    const size_t BUFFER_SIZE = 1024;
    char buffer[BUFFER_SIZE];
    ssize_t bytesRead = 0;

    // 2. Чтение файла
    bytesRead = read(fd, buffer, BUFFER_SIZE - 1);
    if (bytesRead >= 0) {
        buffer[bytesRead] = '\0'; // Терминирующий ноль для строки
        std::cout << "Прочитано байт: " << bytesRead << std::endl;
        std::cout << "Данные: " << buffer << std::endl;
    } else {
        std::cerr << "Ошибка read. Код: " << errno << std::endl;
    }

    // 3. Закрытие дескриптора
    close(fd);

    return 0;
}
```

Консольный вывод

```
Найдено файлов: 1
[*] Обработка: windows_example.cpp
  ↳ Попытка 1
Код успешно прошел проверку g++
Сохранено: OutputCode/windows_example.cpp
```



Задание 2 (Рабочая тетрадь 2)

Задача: Напишите программу, которая демонстрирует безопасную работу с указателями, чтобы избежать пустых ссылок.
Программа должна выделять память для указателя, проверять её успешность, использовать указатель, а затем корректно освобождать память.

Реализация

Для выполнения задания разработана программа на языке **C++**, демонстрирующая безопасное управление динамической памятью:

- Выделение памяти для **одиночного объекта** с проверкой успешности.
- Выделение памяти для **массива** с проверкой.
- Реализация **класса `SafeString`**, который корректно управляет строкой в динамической памяти:
- Конструктор с проверкой `new(std::nothrow)`.
- Глубокая копия в конструкторе копирования и операторе присваивания.
- Деструктор для освобождения памяти.
- Использование **функции `processNumber`**, принимающей указатель и проверяющей его на `nullptr`.
- Во всех случаях память освобождается, а указатели обнуляются.

Программа использует только стандартные библиотеки: `<iostream>`, `<new>`, `<cstring>`.

Принцип работы

1. Выделение памяти для одиночного объекта

```
int* number = new(std::nothrow) int;
if (!number) { /* обработка ошибки */ }
*number = 10;
processNumber(number);  // удваивает значение
delete number;
number = nullptr;
```

- Оператор `new(std::nothrow)` не генерирует исключение, а возвращает `nullptr` при неудаче.
- Функция `processNumber` проверяет переданный указатель на `nullptr`.
- После использования память освобождается, указатель обнуляется.

2. Выделение памяти для массива

```
int* arr = new(std::nothrow) int[5];
if (!arr) { /* обработка ошибки */ }
// заполнение и вывод
delete[] arr;
arr = nullptr;
```

Аналогичные проверки и корректное освобождение через `delete[]`.

3. Класс `SafeString`

- **Конструктор**: выделяет память под копию переданной строки, используя `new(std::nothrow)`. При неудаче выводит сообщение и устанавливает `data = nullptr`.
- **Конструктор копирования**: создаёт глубокую копию, проверяя выделение памяти.
- **Оператор присваивания**: освобождает старую память, затем выделяет новую.
- **Деструктор**: освобождает память и обнуляет указатель.
- **Метод `print`**: выводит строку или сообщение `(пусто)`, если память не выделена.

Такой подход гарантирует отсутствие утечек памяти и безопасную работу даже при сбоях выделения.

Листинг кода

Исходный код: [main.cpp]

```
#include <iostream>
#include <new>
#include <cstring>

class SafeString {
private:
    char* data;
    size_t length;

public:
    // Конструктор
    SafeString(const char* str) : data(nullptr), length(0) {
        if (!str) {
            std::cerr << "Ошибка: передана пустая строка\n";
            return;
        }

        length = std::strlen(str);
        data = new(std::nothrow) char[length + 1];
        if (!data) {
    ...
    ...
    ...
```

```
    return 0;
}
```

Пример работы

Компиляция и запуск (Linux/macOS)

```
g++ -std=c++11 main.cpp -o safe_ptr
./safe_ptr
```

Вывод программы

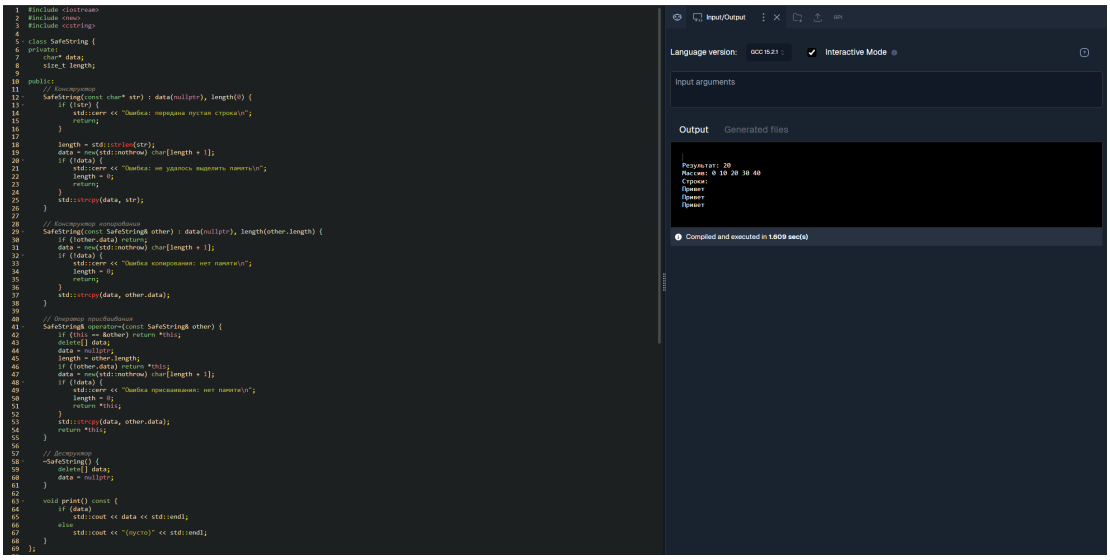
```
Результат: 20
Массив: 0 10 20 30 40
Строки:
Привет
Привет
Привет
```

Пример обработки ошибок (если бы память не выделилась)

При недостатке памяти программа выведет соответствующее сообщение и завершится без аварийного падения.

Скриншоты работы

Вывод в консоли



Задание 3 (Рабочая тетрадь 2)

Задача: Напишите программу, которая выполняет вычисления над большим массивом данных. Используйте параллельное выполнение задач (например, с помощью потоков или асинхронного программирования) для оптимизации производительности. Программа должна разделить задачу на несколько частей и выполнить их одновременно.

Реализация

Для выполнения задания разработана программа на языке **Python**, демонстрирующая преимущества параллельных вычислений на задаче поиска простых чисел.

- Вычисляется количество простых чисел в диапазоне до **5 000 000** (требовательная к

процессору задача).
- Реализованы **два подхода**:
- **Последовательный** – один поток обрабатывает весь диапазон.
- **Параллельный** – диапазон разбивается на равные части, каждая обрабатывается в отдельном процессе (с помощью `ProcessPoolExecutor`).
- Сравнивается время выполнения, вычисляется **ускорение** (speedup).

Используются только стандартные библиотеки: `concurrent.futures`, `time`, `math`.

Принцип работы

1. Функция `is_prime(n)`

Проверяет, является ли число `n` простым: - Отсекает `n < 2`, чётные числа. - Перебирает нечётные делители до √n.

2. Функция `count_primes_in_range(start, end)`

Проходит по числам от `start` до `end-1` и суммирует количество простых.

3. Разделение задачи

Общий диапазон `[0, total_limit)` разбивается на `num_workers` равных интервалов. Например, для `total_limit = 5_000_000` и `num_workers = 4` получаем интервалы: - `[0, 1_250_000)` - `[1_250_000, 2_500_000)` - `[2_500_000, 3_750_000)` - `[3_750_000, 5_000_000)`

4. Последовательное выполнение

Вызывается `count_primes_in_range(0, total_limit)` в главном потоке.

5. Параллельное выполнение

- Создаётся пул процессов `ProcessPoolExecutor(max_workers=num_workers)`.
- Каждый интервал отправляется в отдельный процесс через `executor.submit()`.
- Результаты собираются и суммируются.

6. Вывод результатов

- Количество простых чисел (одинаково для обоих способов).
- Затраченное время.
- Ускорение = `время_последовательно / время_параллельно`.

  Почему процессы, а не потоки?
  Из-за GIL (Global Interpreter Lock) в Python потоки не дают реального параллелизма для CPU-интенсивных задач. `ProcessPoolExecutor` создаёт отдельные процессы, каждый со своим интерпретатором, что позволяет задействовать несколько ядер процессора.

Листинг кода

Исходный код: main.py

```
import concurrent.futures
import time
import math

def is_prime(n):
    if n < 2: return False
    if n == 2: return True
    if n % 2 == 0: return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True
```

```
def count_primes_in_range(start, end):
    count = 0
    for i in range(start, end):
        if is_prime(i):
            count += 1
    return count

def main():

    ...
    ...
    ...

if __name__ == "__main__":
    main()
```

Пример работы

Компиляция и запуск (любая ОС с Python 3.6+)

```
python main.py
```

Скриншоты работы

Последовательный запуск	Параллельный запуск